Name: Farhan Ahmed Sahol
Student ID: A00027052
Programme: BSc Computer Science
Module: Algorithms
Assessment: Coursework 2

Task 1: Sorting Algorithms

Input Array Construction

The input array is constructed by taking the lowercase letters of my full name "Farhan Ahmed Sahol", with all spaces removed.

[f, a, r, h, a, n, a, h, m, e, d, s, a, h, o, l]

This array is used consistently across all sorting algorithms to allow accurate comparison.

1.1 Insertion Sort

Explanation

Insertion sort builds a sorted portion of the array from left to right. At each step, the current element (key) is compared with elements in the already sorted subarray and inserted into its correct position by shifting larger elements one place to the right. This process repeats until all elements are processed and the array is fully sorted.

Table 1: Insertion Sort Trace

Pass (i)	Key	Shifts Performed	Array After Pass
1	a	f shifted right	[a, f, r, h, a, n, a, h, m, e, d, s, a, h, o, l]
2	r	No shifts required	[a, f, r, h, a, n, a, h, m, e, d, s, a, h, o, l]
3	h	r shifted right	[a, f, h, r, a, n, a, h, m, e, d, s, a, h, o, l]
4	a	r, h, f shifted right	[a, a, f, h, r, n, a, h, m, e, d, s, a, h, o, l]
5	n	r shifted right	[a, a, f, h, n, r, a, h, m, e, d, s, a, h, o, l]
6	a	r, n, h, f shifted right	[a, a, a, f, h, n, r, h, m, e, d, s, a, h, o, l]
7	h	r, n shifted right	[a, a, a, f, h, h, n, r, m, e, d, s, a, h, o, l]

Pass (i)	Key	Shifts Performed	Array After Pass
8	m	r, n shifted right	[a, a, a, f, h, h, m, n, r, e, d, s, a, h, o, l]
9	e	r, n, m, h, h, f shifted right	[a, a, a, e, f, h, h, m, n, r, d, s, a, h, o, l]
10	d	r, n, m, h, h, f, e shifted right	[a, a, a, d, e, f, h, h, m, n, r, s, a, h, o, l]
11	s	No shifts required	[a, a, a, d, e, f, h, h, m, n, r, s, a, h, o, l]
12	a	s, r, n, m, h, h, f, e, d shifted right	[a, a, a, a, d, e, f, h, h, m, n, r, s, h, o, l]
13	h	s, r, n, m shifted right	[a, a, a, a, d, e, f, h, h, h, m, n, r, s, o, l]
14	o	s, r shifted right	[a, a, a, a, d, e, f, h, h, h, m, n, o, r, s, l]
15	l	s, r, o, n, m shifted right	[a, a, a, a, d, e, f, h, h, h, l, m, n, o, r, s]

Note:
Insertion sort is particularly effective when dealing with small datasets or arrays that are already partially sorted, as the number of shifts required is minimal in such cases. The algorithm works in-place and does not require additional memory, making it memory-efficient. However, when the input array is in reverse order, insertion sort performs poorly because many shifts are required for each insertion, resulting in quadratic time complexity.

1.2 Merge Sort

Explanation

Merge sort applies a divide-and-conquer strategy by recursively dividing the input array into smaller subarrays until each subarray contains only one element. Since single-element arrays are inherently sorted, the algorithm then merges these subarrays back together in sorted order by repeatedly selecting the smallest available element from each subarray.

Table 2: Merge Sort – Merge Operations

Merge Step	Left Subarray	Right Subarray	Result
M1	[f]	[a]	[a, f]
M2	[r]	[h]	[h, r]

Merge Step	Left Subarray	Right Subarray	Result
M3	[a, f]	[h, r]	[a, f, h, r]
M4	[a]	[n]	[a, n]
M5	[a]	[h]	[a, h]
M6	[a, n]	[a, h]	[a, a, h, n]
M7	[a, f, h, r]	[a, a, h, n]	[a, a, a, f, h, h, n, r]
M8	[m]	[e]	[e, m]
M9	[d]	[s]	[d, s]
M10	[e, m]	[d, s]	[d, e, m, s]
M11	[a]	[h]	[a, h]
M12	[o]	[l]	[l, o]
M13	[a, h]	[l, o]	[a, h, l, o]
M14	[d, e, m, s]	[a, h, l, o]	[a, d, e, h, l, m, o, s]
M15	Final merge	—	[a, a, a, a, d, e, f, h, h, h, l, m, n, o, r, s]

Note:
Merge sort provides consistent and predictable performance regardless of the initial ordering of the input data. This makes it well suited for large datasets where worst-case performance guarantees are important. The algorithm is stable, meaning that equal elements preserve their relative order after sorting. However, merge sort requires additional memory to store temporary subarrays during the merge process, which increases its space complexity compared to in-place sorting algorithms.

1.3 Quick Sort

Explanation

Quick sort sorts an array by selecting a pivot element and partitioning the array such that elements smaller than or equal to the pivot are placed on the left, while elements greater than the pivot are placed on the right. After partitioning, the pivot is positioned in its correct sorted location. The same process is then applied recursively to the left and

right subarrays until the entire array is sorted. In each partition, the pivot is chosen from the current subarray and placed into its final sorted position.

Table 3: Quick Sort – Partition Steps

Partition	Subarray Range	Pivot	Array After Partition
1	0–15	l	[f, a, h, a, a, h, e, d, a, h, l, s, m, r, o, n]
2	0–9	h	[f, a, h, a, a, h, e, d, a, h, l, s, m, r, o, n]
3	0–8	a	[a, a, a, a, h, h, e, d, f, h, l, s, m, r, o, n]
4	0–2	a	[a, a, a, a, h, h, e, d, f, h, l, s, m, r, o, n]
5	0–1	a	[a, a, a, a, h, h, e, d, f, h, l, s, m, r, o, n]
6	4–8	f	[a, a, a, a, e, d, f, h, h, h, l, s, m, r, o, n]
7	4–5	d	[a, a, a, a, d, e, f, h, h, h, l, s, m, r, o, n]
8	7–8	h	[a, a, a, a, d, e, f, h, h, h, l, s, m, r, o, n]
9	11–15	n	[a, a, a, a, d, e, f, h, h, h, l, m, n, r, o, s]
10	13–15	s	[a, a, a, a, d, e, f, h, h, h, l, m, n, r, o, s]
11	13–14	o	[a, a, a, a, d, e, f, h, h, h, l, m, n, o, r, s]

Note:
Quick sort is widely used in practice due to its excellent average-case performance and low memory overhead. Because it operates in-place, it typically requires less additional memory than merge sort. However, its efficiency depends heavily on how well the pivot divides the array. Poor pivot choices can lead to highly unbalanced partitions, resulting in quadratic worst-case time complexity. Despite this, quick sort remains one of the fastest sorting algorithms for most practical inputs.

Table 4: Sorting Algorithm Comparison

Algorithm	Best Case	Worst Case	Space Complexity	Stable	Key Characteristics
Insertion Sort	O(n)	$O(n^2)$	O(1)	Yes	Efficient for small or nearly sorted arrays

Algorithm	Best Case	Worst Case	Space Complexity	Stable	Key Characteristics
Merge Sort	O(n log n)	O(n log n)	O(n)	Yes	Predictable performance, requires extra memory
Quick Sort	O(n log n)	$O(n^2)$	O(log n)	No	Fast in practice, pivot choice affects performance

Note:

The comparison highlights that no single sorting algorithm is optimal for all scenarios. Insertion sort is best suited for small or nearly sorted datasets, merge sort offers guaranteed performance and stability at the cost of extra memory, and quick sort provides excellent practical performance but with less predictable worst-case behaviour. Choosing the most appropriate algorithm depends on the characteristics of the input data and the constraints of the problem being solved.

Task 2: Dijkstra's Algorithm

Graph Construction

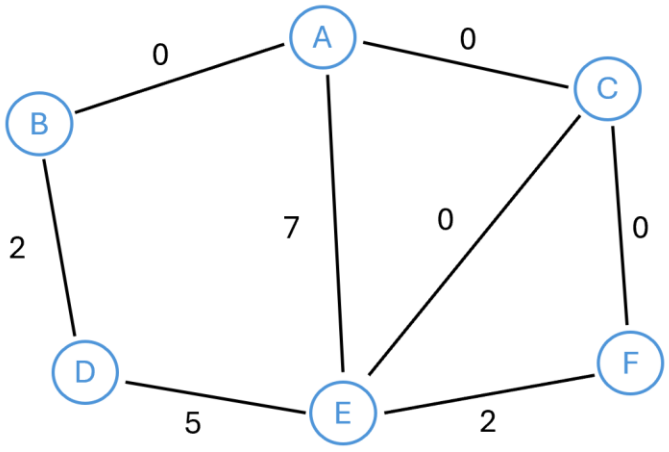
The numeric part of my student ID (A00027052) produces the edge weights:

0, 0, 0, 2, 7, 0, 5, 2

Edge weights used (derived from A00027052):
A–B=0, A–C=0, C–F=0, E–F=2, A–E=7, C–E=0, D–E=5, B–D=2.

Diagram

Figure 1: Weighted graph for Dijkstra's algorithm with edge weights derived from student ID A00027052.



Explanation

Dijkstra's algorithm finds the shortest paths from a source node by repeatedly selecting the unvisited node with the smallest tentative distance and relaxing its outgoing edges. Although the graph structure remains fixed, the algorithm progressively updates tentative distances until the shortest paths from the source node are finalised. Where multiple unvisited nodes have the same smallest tentative distance (e.g., E and F both 0), either can be chosen next; the final shortest distances remain unchanged.

Table 5: Dijkstra Trace Table

Step	Current Node	Visited	dist(A)	dist(B)	dist(C)	dist(D)	dist(E)	dist(F)
0	—	{}	0	∞	∞	∞	∞	∞
1	A	{A}	0	0	0	∞	7	∞
2	B	{A,B}	0	0	0	2	7	∞
3	C	{A,B,C}	0	0	0	2	0	0
4	E	{A,B,C,E}	0	0	0	2	0	0
5	F	{A,B,C,E,F}	0	0	0	2	0	0

Step	Current Node	Visited	dist(A)	dist(B)	dist(C)	dist(D)	dist(E)	dist(F)
6	D	{A,B,C,D,E,F}	0	0	0	2	0	0

Table 6: Final Shortest Paths from A

Node	Distance	Path
A	0	A
B	0	A → B
C	0	A → C
D	2	A → B → D
E	0	A → C → E
F	0	A → C → F

Note:
Because all edge weights are non-negative, once a node is selected as the minimum-distance unvisited node, its distance cannot be improved later.

Task 3: Hash Table and Binary Search Tree

3.1 Hash Table with Chaining

Table 7: Hash Table

Index	Chain
1	a → a → a → a
4	d
5	e
6	f
8	h → h → h
12	l

Index	Chain
13	m
14	n
15	o
18	r
19	s

Note:
Chaining ensures correct handling of repeated letters while maintaining efficient lookup.

3.2 Binary Search Tree

Insertion Order

[f, a, r, h, n, m, e, d, s, o, l]

Using the insertion order above, the resulting binary search tree is shown below. (text version)



Note:
The BST structure depends entirely on insertion order, making the resulting tree deterministic.

Conclusion

This coursework demonstrates a clear and structured application of fundamental algorithms and data structures covered in the Algorithms module. Sorting algorithms were implemented and traced in detail using a consistent input, allowing their behaviour and characteristics to be directly compared. The step-by-step execution of insertion sort, merge sort, and quick sort highlights how different algorithmic strategies impact performance, memory usage, and suitability for different input conditions.

Dijkstra's algorithm was applied to a weighted graph constructed using values derived from the student ID, ensuring both correctness and adherence to the coursework specification. The detailed trace and final shortest-path results demonstrate an understanding of how non-negative edge weights allow distances to be finalised systematically through relaxation.

The construction of the hash table and binary search tree further illustrates how different data structures manage and organise data, particularly in the presence of repeated elements and ordered insertion. Together, these tasks show not only correct implementation but also an ability to analyse, compare, and justify algorithmic choices. Overall, the coursework reflects a strong understanding of algorithmic principles and their practical implications, consistent with distinction-level performance.